

Ejercicios de estructuras de datos avanzadas

(listas, pilas, colas, tablas hash y grafos)

- 1) Implementa una clase llamada "Cliente" con estos atributos de tipo String: nombre, e-mail. Ambos se pasarán por el constructor, añade también setters y getters. Por otra parte, necesitamos un campo para albergar los teléfonos, llámalo como quieras y elige el tipo que consideres más adecuado (LinkedList o ArrayList) de String. También necesitamos un método "guardarTelefono" que hace lo evidente, sin repetidos; y otro método "getTelefono" que devuelve el teléfono que hay en el índice pasado por parámetro. Finalmente tendremos el método "buscarClientePorTelefono" al que le pasaremos una cadena conteniendo un teléfono y nos devolverá el nombre del cliente al que pertenece dicho número. Prueba tu aplicación a partir de un programa principal.
- 2) Implementa una aplicación que lea de un fichero de texto (que contenga algunas líneas) e imprima dichas líneas en consola, pero en orden inverso; para ello, tendrás que decidir qué estructura de datos es más apropiada. Si el fichero contiene: A (salto) B (salto) C; se imprimirá: C (salto) B (salto) A
- 3) Implementa un método estático llamado "insertarOrdenado" que reciba un entero como parámetro y lo "inserte" en una pila de enteros, pero de forma ordenada. Para ello tendrás que usar la clase Stack y los métodos apropiados, así como la lógica de programación necesaria. Implementa un programa de ejemplo para probar el funcionamiento.
- 4) Implementa un método estático que determine si los delimitadores: (,) , { , } , [,] utilizados en expresiones matemáticas (por ejemplo: $[(7+2x)*(2y-4z)]$) están equilibrados. Recibirá un string con la expresión y devolverá un booleano.

Pistas: si encontramos un símbolo de apertura lo apilamos. Si encontramos un símbolo de cierre, consultamos el elemento que hay en la cima. Si son de distinto tipo la expresión no está balanceada. Si son del mismo tipo, debemos desapilar y seguir. La expresión estará balanceada si al terminar de procesar la cadena, la pila está vacía.
- 5) Implementa una cola de clientes en la caja de un supermercado, para ello sólo almacenaremos los nombres de los clientes (String). Necesitamos un método para echar a la calle a un cliente si se pone nervioso y molesta al resto, el método se llamará "expulsarCliente". Recibirá un String con el nombre del cliente a expulsar y lo eliminará de la cola manteniendo al resto en su lugar.
- 6) Se desea implementar un historial de navegador donde se va a registrar la cantidad de veces que una URL ha sido visitada, para ello, se tendrá que usar una tabla hash cuyo par clave-valor será String-Integer (siendo String la dirección web). La aplicación solicitará al usuario que escriba la dirección y actuará en consecuencia. También debe

permitir consultar, por consola, el contenido del historial, así como borrar selectivamente parte o todo.